## PART C – BIG-O ALGORITHMIC COMPLEXITY ANALYSIS

| Operation | Best | Average | Worst | Space | Explanation / Refactored Routine |
|---|---|---|---|---|---|
| Unamortized Insert – add_item() / addItem() | O(1) | O(1) | O(n) | O(1) | Normal insertion writes one element. When resize is needed, n existing elements are copied, making that insertion O(n). The refactored code uses addItem() with automatic resize. |
| Linear Deletion – remove_item_at() / removeItemAt() | O(1) | O(n) | O(n) | O(1) | Removing near the end needs little/no shifting, while removing earlier elements shifts remaining items. The refactored code performs one shift loop. |
| Linear Search – findItem() | O(1) | O(n) | O(n) | O(1) | The first item may match immediately; otherwise the array may be scanned until the target is found or all elements are checked. |
| Array Display – printAll() | O(n) | O(n) | O(n) | O(1) | Every valid element is visited once. The refactored code loops from 0 while i < size. |
| Matrix Transpose – processMatrix() | O(R×C) | O(R×C) | O(R×C) | O(R×C) | The nested loops visit every matrix cell. The supplied code uses a fixed 3×3 example, so the actual demonstration is constant-sized, while the activity sheet expresses it |

| | | | | | |
|---|---|---|---|---|---|
| | | | | | generally as R×C. |